

Relatório Técnico - Programa Identifier

Teste funcional, classes de equivalência e análise de valor limite

Data da execução: 14/09/2026

1. Objetivo e base documental

Este relatório documenta a implementação e a validação funcional do programa Identifier. O trabalho aplica Particionamento em Classes de Equivalência e Análise de Valor Limite, implementa os casos em JUnit e registra os resultados obtidos na aplicação final.

O documento `Aula02ATV03PlanoDeTeste/ativ03/Identifier_Testes_de_Software (1).md` foi utilizado como base. A lista de 16 casos da atividade anterior é mantida na Seção 5 para comparação com a nova lista sistemática.

2. Especificação e interpretação adotada

Um identificador válido deve cumprir simultaneamente estas regras:

- possuir no mínimo 1 e no máximo 6 caracteres;
- começar com uma letra ASCII maiúscula ou minúscula (A-Z ou a-z);
- conter, nas demais posições, somente letras ASCII ou dígitos (0-9).

A interpretação ASCII é a mesma adotada pelo documento-base. Espaços, pontuação, símbolos, letras acentuadas e outros caracteres Unicode não pertencem às classes válidas. A API também rejeita `null` de forma segura.

3. Passo 1 - Identificação das classes de equivalência

As condições foram separadas por atributo. Como as regras são cumulativas, uma entrada só é válida quando pertence a todas as classes válidas aplicáveis.

ID	Atributo	Tipo	Definição	Exemplos
C-CV	Comprimento	Válida	1 a 6 caracteres	A, abc, a1b2c3
C-CI1	Comprimento	Inválida	0 caracteres	string vazia ""
C-CI2	Comprimento	Inválida	7 ou mais caracteres	abcdefg
C-IV	Inicial	Válida	Letra entre A-Z ou a-z	A, abc
C-II1	Inicial	Inválida	Dígito entre 0-9	1abc
C-II2	Inicial	Inválida	Símbolo ou pontuação	_abc
C-II3	Inicial	Inválida	Espaço em branco	" abc"
C-DV	Demais caracteres	Válida	Letras ou dígitos; a classe não se aplica quando o comprimento é 1	a1B
C-DI1	Demais caracteres	Inválida	Contém símbolo ou pontuação	a#b
C-DI2	Demais caracteres	Inválida	Contém espaço em branco	"a b"

Cada classe inválida recebe um representante que mantém as outras condições válidas sempre que possível. Isso permite associar a rejeição a uma regra específica, característica do particionamento fraco normal.

4. Passo 2 - Nova lista de casos derivados

ID	Entrada	Classes cobertas	Resultado esperado	Resultado obtido
CE01	abc	C-CV, C-IV, C-DV	Válido	Válido - passou
CE02	""	C-CI1	Inválido	Inválido - passou
CE03	abcdefg	C-CI2	Inválido	Inválido - passou
CE04	1abc	C-II1	Inválido	Inválido - passou
CE05	_abc	C-II2	Inválido	Inválido - passou
CE06	" abc"	C-II3	Inválido	Inválido - passou
CE07	a#b	C-DI1	Inválido	Inválido - passou
CE08	"a b"	C-DI2	Inválido	Inválido - passou
AVL01	A	C-CV, C-IV; limite 1	Válido	Válido - passou
AVL02	a1b2c3	C-CV, C-IV, C-DV; limite 6	Válido	Válido - passou

4.1 Complemento pela análise de valor limite

Tamanho	Relação com a fronteira	Entrada	Esperado	Caso automatizado
0	mínimo menos 1	""	Inválido	CE02
1	mínimo permitido	A	Válido	AVL01
6	máximo permitido	a1b2c3	Válido	AVL02
7	máximo mais 1	abcdefg	Inválido	CE03

Os tamanhos 0 e 7 já representam classes inválidas de comprimento. Os tamanhos

1 e 6 exercitam precisamente as duas fronteiras válidas e, por isso,

complementam o representante intermediário `abc` escolhido no particionamento.

5. Lista da atividade anterior mantida para comparação

Os resultados abaixo são o registro histórico do documento-base, não o resultado da implementação final desta atividade.

ID	Entrada	Objetivo	Esperado	Resultado histórico
H01	A	Comprimento mínimo	Válido	Sucesso
H02	a1b2c3	Comprimento máximo	Válido	Sucesso
H03	Ab	Comprimento curto	Válido	Sucesso
H04	acbde	Somente letras	Válido	Sucesso
H05	x12	Letras e dígitos	Válido	Sucesso
H06	" "	Entrada com um espaço	Inválido	Crash registrado
H07	abcdefg	Sete caracteres	Inválido	Aceito indevidamente
H08	stringmuitogrande	Entrada muito grande	Inválido	Sucesso
H09	1abc	Inicia com dígito	Inválido	Sucesso
H10	_abc	Inicia com especial	Inválido	Sucesso
H11	" a"	Inicia com espaço	Inválido	Sucesso
H12	a#b	Especial interno	Inválido	Aceito indevidamente
H13	"a b"	Espaço interno	Inválido	Sucesso
H14	a.b	Pontuação interna	Inválido	Sucesso
H15	sem argumento	Interface de console	Mensagem de uso	Sucesso
H16	arg1 arg2	Múltiplos argumentos	Erro ou ignorar extra	Sucesso

Duas imprecisões do material anterior foram removidas na nova lista. O caso H06 usa um espaço e não uma string vazia, portanto não representa comprimento zero. Além disso, H16 tinha duas respostas aceitáveis; a implementação atual fixa o contrato em exibir a mensagem de uso sempre que a quantidade de argumentos for diferente de um.

6. Implementação e automação dos testes

O programa foi implementado em Java 21. O método público

`validateIdentifier(String)` verifica primeiro a nulidade e o comprimento,

depois a letra inicial e, por fim, percorre os demais caracteres. A interface de

console aceita exatamente um argumento e imprime *Válido*, *Inválido* ou a

mensagem de uso.

Os testes usam JUnit Jupiter 5.14.3 e Maven. Todos seguem a estrutura formal:

- **Setup:** uma nova instância do validador ou uma saída de console capturável é configurada antes do teste;
- **Invocation:** o método de validação ou o programa principal é executado com a entrada do caso;
- **Assessment:** `assertEquals` ou `assertFalse` compara o valor observado ao esperado e identifica o caso em uma eventual falha.

Suíte	Cobertura	Testes executados	Falhas	Erros
IdentifierLegacyTest	H01 a H14	14	0	0
IdentifierEquivalenceTest	CE01 a CE08, AVL01, AVL02 e ROB01	11	0	0
IdentifierMainTest	CLI01, CLI02, H15 e H16	4	0	0
Total	Validador e console	29	0	0

Comando de execução: `.\mvnw.cmd clean test`

Resultado geral: 29 testes aprovados (100%), sem falhas, erros ou testes ignorados.

O Maven registrou `BUILD SUCCESS` em 14/09/2026. Os relatórios detalhados foram gerados em `target/surefire-reports`.

7. Conclusão

O particionamento tornou explícitas dez classes de entrada e garantiu ao menos um representante por classe. A análise de valor limite acrescentou verificações

precisas nas fronteiras 1 e 6 e reforçou as fronteiras inválidas 0 e 7. Em

comparação com a lista anterior, a nova abordagem removeu a confusão entre

espaço e string vazia e definiu uma única resposta para múltiplos argumentos.

A implementação final corrigiu os três comportamentos problemáticos registrados

historicamente: entrada composta por espaço não causa interrupção, sete

caracteres são rejeitados e caracteres especiais internos não são aceitos. A

execução automatizada de 29 testes terminou com taxa de aprovação de 100%.